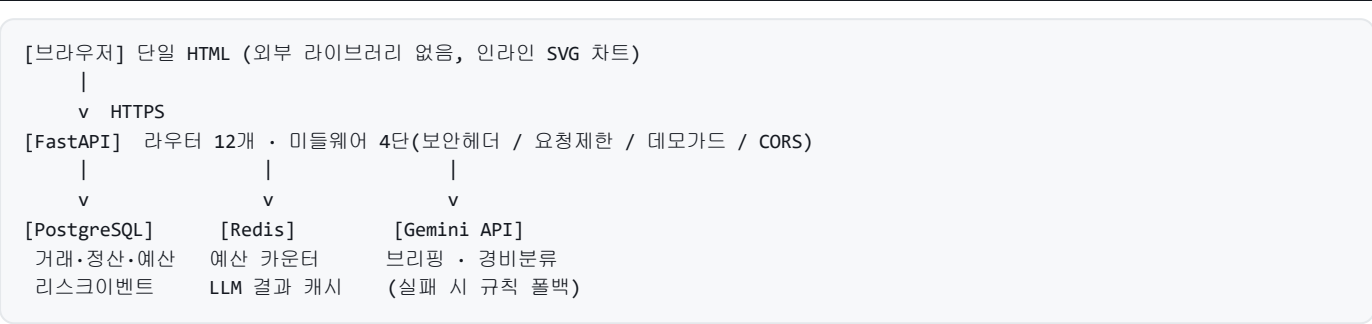


시스템 설계

구성



API 목록 (코드에서 자동 추출, 25개)

메서드	경로	모듈	설명
GET	/api/v1/brief/{user_id}	brief	
POST	/api/v1/budgets	budgets	카테고리별 월 예산 한도를 설정합니다. 이미 있으면 한도만 갱신합니다.
GET	/api/v1/budgets/usage	budgets	유저가 설정해둔 모든 카테고리의 이번 기간 사용량을 반환합니다.
POST	/api/v1/budgets/record-spending	budgets	
GET	/api/v1/budgets/forecast	budgets	
GET	/api/v1/demo/personas	demo	
GET	/api/v1/expense/classify	expense	해당 유저의 최근 거래 <limit>건을 실제로 분류해서 스와이프 큐로 반환합니다.
GET	/api/v1/gig-score/{user_id}	gig_score	
GET	/api/v1/guidance/{user_id}	guidance	
GET	/api/v1/income/sources	income	이 유저에게 연결 가능한 플랫폼 전체 목록. 연결 안 된 것도 포함해서 보여줍니다
POST	/api/v1/income/sources/{source_id}/connect	income	플랫폼 연결하기. 이미 연결돼있으면 그냥 현재 상태를 그대로 돌려줍니다(재시도해도 안전).
POST	/api/v1/income/sources/{source_id}/disconnect	income	연결 해제. 실제 계좌 연동이라면 이 시점에 마이데이터 접근 권한을 회수하는
GET	/api/v1/income/summary	income	연결된 플랫폼만 집계해서 '한 장 요약' 홈 카드용 데이터를 만듭니다.
GET	/api/v1/income/events	income	최근 수입 이벤트 목록 (연결된 플랫폼만, 최신순).

POST	/api/v1/risk/assess	risk	
GET	/api/v1/risk/events	risk	
GET	/api/v1/risk/score/{user_id}	risk	
GET	/api/v1/shield/{user_id}	shield	
GET	/api/v1/shield/{user_id}/scenarios	shield	
GET	/api/v1/shield/{user_id}/settings	shield	
PUT	/api/v1/shield/{user_id}/settings	shield	
POST	/api/v1/simulate/cashflow	simulate	
GET	/api/v1/tax/estimate/{user_id}	tax	
GET	/api/v1/validation/gemini	validation	어떤 Gemini 모델이 실제로 응답하는지 하나씩 찢어보는 진단용 엔드포인트.
GET	/api/v1/validation/forecast	validation	예측 엔진의 홀드아웃 검증 결과. 시연 인물 전체를 대상으로 계산합니다.

데이터 모델

users

user_id	VARCHAR(50) PRIMARY KEY,
name	VARCHAR(100) NOT NULL,
persona	VARCHAR(100), -- 예: '김지훈-프리랜서', '김민철-생계형투잡러'
created_at	TIMESTAMP NOT NULL DEFAULT now()

user_risk_profiles

user_id	VARCHAR(50) PRIMARY KEY REFERENCES users(user_id),
avg_transaction_amount	NUMERIC(14, 2) DEFAULT 0,
std_transaction_amount	NUMERIC(14, 2) DEFAULT 0,
single_transaction_limit	NUMERIC(14, 2) DEFAULT 500000, -- 단일 거래 한도(기본값)
daily_limit	NUMERIC(14, 2) DEFAULT 1000000, -- 일일 누적 한도(기본값)
avg_daily_transactions	NUMERIC(10, 2) DEFAULT 0,
allowed_countries	TEXT[] DEFAULT ARRAY['KR'], -- 허용 국가 코드 목록
updated_at	TIMESTAMP NOT NULL DEFAULT now()

transactions

transaction_id	UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id	VARCHAR(50) NOT NULL REFERENCES users(user_id),
amount	NUMERIC(14, 2) NOT NULL,
merchant_id	VARCHAR(50),
merchant_name	VARCHAR(200),
mcc_code	VARCHAR(10),
category	VARCHAR(50), -- C(하은)의 카테고리 분류기가 채워주는 필드
timestamp	TIMESTAMP NOT NULL,
latitude	DOUBLE PRECISION,
longitude	DOUBLE PRECISION,
country	VARCHAR(5) DEFAULT 'KR',
device_id	VARCHAR(100),
created_at	TIMESTAMP NOT NULL DEFAULT now()

risk_events

```
event_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
transaction_id UUID REFERENCES transactions(transaction_id),
user_id       VARCHAR(50) NOT NULL REFERENCES users(user_id),
rule_id       VARCHAR(20),          -- 예: 'R001' ~ 'R007'
severity      VARCHAR(20),          -- LOW / MEDIUM / HIGH / CRITICAL
description   TEXT,
action        VARCHAR(30),          -- ALLOW / FLAG_FOR_REVIEW / REQUIRE_CONFIRMATION / BLOCK
score         INTEGER,              -- 0~100 종합 리스크 점수
resolved      BOOLEAN DEFAULT FALSE,
created_at    TIMESTAMP NOT NULL DEFAULT now()
```

budgets

```
budget_id     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id       VARCHAR(50) NOT NULL REFERENCES users(user_id),
category      VARCHAR(50) NOT NULL,
period        VARCHAR(20) NOT NULL DEFAULT 'monthly', -- weekly/monthly/quarterly
limit_amount  NUMERIC(14, 2) NOT NULL,
alert_thresholds NUMERIC(4, 2)[] DEFAULT ARRAY[0.5, 0.8, 0.9, 1.0, 1.2],
rollover_enabled BOOLEAN DEFAULT FALSE,
created_at    TIMESTAMP NOT NULL DEFAULT now(),
updated_at    TIMESTAMP NOT NULL DEFAULT now(),
UNIQUE(user_id, category, period)
```

category_mapping

```
keyword       VARCHAR(100) PRIMARY KEY,
category      VARCHAR(50) NOT NULL
```

income_sources

```
source_id     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id       VARCHAR(50) NOT NULL REFERENCES users(user_id),
platform_name VARCHAR(100) NOT NULL,    -- '배달의민족', '크몽', '유튜브' 등
platform_type VARCHAR(30) NOT NULL,     -- '배달' / '프리랜서' / '콘텐츠' / '커머스' / '본업'
icon_emoji    VARCHAR(10) NOT NULL DEFAULT '👉',
connected     BOOLEAN NOT NULL DEFAULT FALSE,
connected_at  TIMESTAMP,
created_at    TIMESTAMP NOT NULL DEFAULT now(),
UNIQUE (user_id, platform_name)
```

income_events

```
income_id     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
source_id     UUID NOT NULL REFERENCES income_sources(source_id),
user_id       VARCHAR(50) NOT NULL REFERENCES users(user_id),
amount        NUMERIC(14, 2) NOT NULL,
memo          VARCHAR(200),
status        VARCHAR(20) NOT NULL DEFAULT '정산완료', -- 정산완료 / 정산예정
settled_at    TIMESTAMP NOT NULL,
created_at    TIMESTAMP NOT NULL DEFAULT now()
```

user_settings

```
user_id       VARCHAR(50) PRIMARY KEY REFERENCES users(user_id),
min_safety_balance NUMERIC(14, 2),
current_balance NUMERIC(14, 2),
updated_at    TIMESTAMP NOT NULL DEFAULT now()
```

핵심 알고리즘

1. 45일 현금흐름 예측 (app/cashflow.py)

- **수입 투사:** 플랫폼별 정산 간격의 **중앙값**과 최근 4회 평균 금액으로 다음 입금을 배치. 표본이 적어 이상치에 강한 중앙값을 택했다.
- **지출 투사:** 반복 결제를 탐지해(간격 중앙값의 $\pm 25\%$ 이내, 5~40일) 미래 지출로 배치하고, 나머지는 최근 60일 지출을 일 단위로 균등 배분.
- **부족일 특징:** 시작 잔액에서 하루씩 누적해 최소 안전잔액을 처음 밑도는 날짜를 찾는다.
- **시나리오:** 정산 10일 지연 / 수입 30% 감소를 적용해 부족 시점의 변화를 계산.

2. 세금·건강보험료 추정 (app/tax.py)

연 사업소득(90일 실적 $\times 4$)
→ 단순경비율 차감 (업종별)
→ 기본공제 150만원
→ 과세표준
→ 8단계 누진세율 적용 (6%~45%)
→ 지방소득세 10% 가산
→ 기납부 원천징수 3.3% 차감
= 5월 정산액 (음수면 환급)

건강보험료 = 대상소득 $\times 7.19\% \div 12$ + 장기요양보험료(건보료 $\times 12.95\%$)
· 직장가입자: 급여 외 소득 중 2,000만원 초과분만 대상 (전액 본인 부담)
· 지역가입자: 소득 전체 대상
· 2,000만원 경계 도달 전 남은 금액을 함께 경고

3. 이상거래 탐지 (app/rule_engine.py + app/anomaly_model.py)

- **규칙 7종:** 단건 한도(R001) · 일일 누적 한도(R002) · 신규 가맹점(R003) · 이동 속도(R004) · 심야 고액(R005) · 단기 반복(R006) · 허용되지 않은 국가(R007)
- **Isolation Forest:** 금액 z-score, 신규 기기 여부, 직전 거래와의 거리, 가맹점 희소성 4개 피처. 규칙이 설계상 잡지 못하는 유형(위치 급변·신규 기기)을 담당
- **앙상블:** 확률적 OR — $1 - (1 - \text{규칙확률}) \times (1 - \text{이상확률})$. 규칙이 이미 최고 위험이면 결과는 그대로 유지된다

4. 생성형 AI 적용 (app/gemini_client.py)

- **브리핑:** 계산이 끝난 수치만 전달하고 2~3문장 설명과 권고 한 줄을 생성. 새로운 숫자를 만들지 말라고 지시해 환각을 차단
- **경비 분류:** 직업·가맹점·시간대·금액으로 인정 가능성(%)과 태그를 생성
- **장애 대응:** 모델 체인을 순서대로 시도하고, 실패 시 쿨다운을 두어 지연을 제한하며, 최종 실패 시 규칙·템플릿으로 자동 강등. 결과는 Redis에 영구 캐싱

보안 조치

항목	내용
SQL 인젝션	전 쿼리 SQLAlchemy <code>text()</code> + 바인드 파라미터. 문자열 조합으로 값을 끼워넣는 곳 없음
XSS	프론트의 모든 동적 텍스트에 <code>escapeHtml()</code> 적용 (LLM 출력이 들어오는 필드가 있어 특히 중요)

접근 제어	DemoUserGuardMiddleware 가 경로-쿼리의 user_id를 검사하고, 본문으로 받는 엔드포인트는 라우터에서 직접 확인
보안 헤더	CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, HSTS(HTTPS 시). 처리되지 않은 500 응답에도 부착
요청 제한	IP당 분당 120회 (/api/v1/*)
컨테이너	비루트 사용자(appuser)로 실행
비밀 관리	.env 는 커밋 제외. 키는 배포 환경변수로만 주입
의존성	pip-audit로 취약점 35건 확인 후 패치